Order_ID	Customer_ID	Sales_Amount	Order_Date
O101	C001	4500	12-01-2024
O102	C002	Null	15-01-2024
O103	C003	3200	2024/01/18
O101	C001	4500	12-01-2024
O104	C004	Three Thousand	20-01-2024
O105	C005	5100	25-01-2024

1. **Identify all data quality issues present in the dataset that can cause problems during data loading.**

1. Duplicate Records

- Row with Order_ID = O101 appears twice.
- This can violate primary key constraints and cause duplication in analysis.

2. Missing Values

- Sales_Amount = NULL for Order_ID O102.
- Missing numeric values can break calculations (e.g., total sales).

3. Inconsistent Data Formats (Date)

- Dates are not in a uniform format:
  - 12-01-2024 (DD-MM-YYYY)
  - 2024/01/18 (YYYY/MM/DD)
- This can cause parsing errors during loading.

4. Invalid Data Type

- Sales_Amount = "Three Thousand" (text instead of numeric) for O104.
- This will fail when loading into numeric columns.

5. Lack of Standardization

- Date format inconsistency (mixed separators - and /).
- Could lead to incorrect sorting or filtering.

6. Potential Integrity Issue

- If Order_ID is expected to be unique (primary key), duplicates violate entity integrity.

2. **Primary Key Validation Assume is the Primary Key. a) Is the dataset violating the Primary Key rule? b) Which record(s) cause this violation?**

a)Yes, the dataset violates the Primary Key constraint because a primary key must be:

- Unique
- Not NULL

b)Here, uniqueness is violated.

The violation is caused by duplicate entries of:

- **Order_ID = O101**

3. **Missing Value Analysis Which column(s) contain missing values? a) List the affected records b) Explain why loading these records without handling missing values is risky**

Sales_Amount column contains missing values.

a)affected records:

Order_ID | Customer_ID | Sales_Amount | Order_Date

O102 | C002 | NULL | 15-01-2024

b) Loading this data as-is can create several problems:

Incorrect Aggregations

- Functions like SUM(), AVG() may give wrong or misleading results.

Data Integrity Issues

- Sales amount is a critical business field; missing values reduce reliability.

Reporting Errors

- Dashboards and reports may show incomplete or inaccurate insights.

Processing Failures

- Some systems or pipelines may reject records with NULL in important fields.

Business Impact

- Decisions based on incomplete sales data can lead to wrong conclusions.

4. **Data Type Validation Identify records where Sales_Amount violates expected data type rules. a) Which record(s) will fail numeric validation? b) What would happen if this dataset is loaded into a SQL table with as DECIMAL?**

a) The **Sales_Amount** column is expected to be numeric, but the following record violates this:

Order_ID | Customer_ID | Sales_Amount | Order_Date

O104 | C004 | Three Thousand | 20-01-2024

b) If we try to load this dataset into a SQL table where **Sales_Amount is defined as DECIMAL**, the following issues may occur:

Data Load Failure (Most Common)

- SQL will throw an error like:
  - *"Invalid numeric value"*
- The entire insert/update may fail depending on the system.

Row Rejection

- In ETL tools, this row may be **rejected and sent to an error table**.

Implicit Conversion Issues (in some systems)

- Some databases may convert invalid text to:
  - NULL or 0 (not reliable, risky behavior)

Data Integrity Risk

- Wrong or missing values can affect:
  - Revenue calculations
  - Reports
  - Business decisions

5. **Date Format Consistency The column has multiple formats. a) List all date formats present in the dataset b) Why is this a problem during data loading?**

a) Date formats present in the dataset
From the Order_Date column:
DD-MM-YYYY
- 12-01-2024
- 15-01-2024
- 20-01-2024
- 25-01-2024

YYYY/MM/DD
- 2024/01/18

b) Mixed date formats can cause several issues:

Parsing Errors
- SQL/ETL tools may fail to recognize multiple formats and throw errors.

Incorrect Date Interpretation
- Example: 12-01-2024 could be interpreted as:
  - 12 Jan 2024 (DD-MM-YYYY)
  - Dec 1, 2024 (MM-DD-YYYY)
- This leads to **wrong data storage**.

Load Failures
- Strict schemas (e.g., DATE type in SQL) may reject inconsistent formats.

Sorting & Filtering Issues
- Dates stored as text or misinterpreted will break:
  - Time-based analysis
  - Monthly/yearly reports

Data Integration Problems
- Combining with other datasets becomes unreliable.

6. **Load Readiness Decision Based on the dataset condition: a) Should this dataset be loaded directly into the database? (Yes/No) b) Justify your answer with at least three reasons**

a) No, the dataset should not be loaded directly.

b) There are multiple data quality issues that make it unsafe to load:

Primary Key Violation

- Duplicate **Order_ID (O101)** exists → breaks uniqueness constraint.

Missing Values

- **Sales_Amount is NULL** for one record → leads to incomplete and unreliable data.

Invalid Data Type

- "Three Thousand" is text → will fail when inserted into a numeric (DECIMAL) column.

Inconsistent Date Formats

- Mixed formats (DD-MM-YYYY and YYYY/MM/DD) → can cause parsing errors or wrong date storage.

Risk of Load Failure

- Database may reject rows or fail the entire batch during insertion.

7. **Pre-Load Validation Checklist List the exact pre-load validation checks you would perform on this dataset before loading**
Q7. Pre-Load Validation Checklist
Before loading this dataset into the database, these pre-load validation checks should be performed:

1. Primary Key Validation
Ensure Order_ID is:
   - Unique (no duplicates like O101)
   - Not NULL

2. Duplicate Record Check
Identify and remove duplicate rows.
Keep only one valid record per Order_ID.

3. Missing Value Check
Detect NULL values in important columns:
   - Sales_Amount

Decide action:
   - Impute / replace / remove records

4. Data Type Validation
Ensure correct data types:
   - Sales_Amount → Numeric
   - Reject/clean values like "Three Thousand"

5. Date Format Standardization
Convert all Order_Date values to a single format:
   - Recommended: YYYY-MM-DD

Validate that all dates are valid calendar dates.

6. Range & Business Rule Validation
Check:
   - Sales_Amount > 0

Prevent invalid values like negative or zero sales.

7. Format Consistency Check
Ensure:
   - No mixed formats (dates, numbers)

- No unwanted characters or text in numeric fields

8. Referential Integrity Check (if applicable)
Validate Customer_ID exists in master table (if available)

9. Data Deduplication Logic
Define rule:
- Keep latest / first / most accurate record when duplicates exist

10. Schema Validation
Ensure dataset matches target table structure:
- Column names
- Data types
- Constraints (PRIMARY KEY, NOT NULL, etc.)

8. **Cleaning Strategy Describe the step-by-step cleaning actions required to make this dataset load-ready.**

Step 1: Remove Duplicate Records
Identify duplicate Order_ID (O101).
Keep only one valid record (exact duplicate → safe to remove).

Step 2: Handle Missing Values
For Sales_Amount = NULL (O102):
- Option 1: Impute (e.g., average/median sales)
- Option 2: Replace with 0 (if business allows)
- Option 3: Remove record (if critical field)

Best practice: Impute or flag, don't blindly delete.

Step 3: Fix Invalid Data Types
Convert "Three Thousand" → 3000 for O104.
Ensure entire Sales_Amount column is numeric.

Step 4: Standardize Date Format
Convert all dates to a consistent format:
- Recommended: YYYY-MM-DD

Example:
- 12-01-2024 → 2024-01-12
- 2024/01/18 → 2024-01-18

Step 5: Validate Business Rules
Ensure:
- Sales_Amount > 0

Remove or correct invalid values if found.

Step 6: Enforce Primary Key Integrity
Ensure:
- Order_ID is unique and NOT NULL

Recheck after duplicate removal.

Step 7: Trim & Clean Formatting Issues
Remove:
- Extra spaces
- Special/unwanted characters

Ensure consistent formatting across all columns.

Step 8: Final Validation Check
Re-run checks:
- No NULLs in critical fields
- Correct data types
- Consistent formats
- No duplicates

Step 9: Load to Database
Load into table with proper schema:
- Order_ID → PRIMARY KEY
- Sales_Amount → DECIMAL
- Order_Date → DATE

9. **Loading Strategy Selection Assume this dataset represents daily sales data. a) Should a Full Load or Incremental Load be used? b) Justify your choice.**
a)Incremental Load should be used.
b) Since this dataset represents daily sales data, loading strategy should be efficient and scalable:
New Data is Added Regularly
- Daily sales means new records come in every day.
- No need to reload historical data repeatedly.

Performance Efficiency
- Incremental load processes only new or changed records.
- Faster and less resource-intensive than full load.

Avoids Redundancy
- Full load would reinsert existing data, increasing:
  - Processing time
  - Risk of duplicates

Supports Real-Time / Near Real-Time Systems
- Incremental loads are ideal for pipelines that run daily or hourly.

Better for Large Datasets
- As data grows, full load becomes slow and impractical.

10. **Assume this dataset was loaded without cleaning and connected to a BI dashboard. a) What incorrect results might appear in Total Sales KPI? b) Which records specifically would cause misleading insights? c) Why would BI tools not detect these issues automatically?**
a) What incorrect results might appear in Total Sales KPI?
If the dataset is loaded without cleaning, the Total Sales KPI will be inaccurate due to:

- Overestimation
  - Duplicate record (O101) will be counted twice → inflated total.
- Underestimation
  - Missing value (NULL in O102) will be ignored in aggregation → lower total.
- Exclusion of Invalid Data

    - Text value ("Three Thousand" in O104) may:
        - Be ignored
        - Treated as NULL
          → further underreporting.
- Inconsistent Calculations
    - Depending on the BI tool, totals may vary due to parsing issues.

b) Which records specifically cause misleading insights?

- O101 (Duplicate)
    - Causes double counting → inflated sales.
- O102 (Missing Sales_Amount)
    - Causes loss of revenue data → underreported sales.
- O104 (Invalid Data Type: "Three Thousand")
    - Causes:
        - Exclusion from calculation OR
        - Conversion errors
          → inaccurate totals.

c) Why would BI tools not detect these issues automatically?

- No Built-in Data Validation
    - BI tools focus on visualization, not deep data cleaning.
- Assume Data is Already Clean
    - They rely on upstream ETL processes for quality.
- Silent Handling of Errors
    - NULLs are ignored in aggregations (e.g., SUM).
    - Invalid values may be skipped without warning.
- Lack of Business Context
    - Tools don’t know:
        - Order_ID should be unique
        - Sales_Amount must be numeric
- Limited Constraint Enforcement
    - Unlike databases, BI tools don’t enforce:
        - Primary keys
        - Data types strictly